



Cambridge (CIE) IGCSE Computer Science



Your notes

Number Systems

Contents

- * Computers & Binary
- * Number Systems
- * Converting Between Binary & Denary
- * Converting Between Hexadecimal & Denary
- * Converting Between Hexadecimal & Binary
- * Uses of Hexadecimal
- * Binary Addition
- * Binary Shifts
- * Two's Complement

Computers & Binary

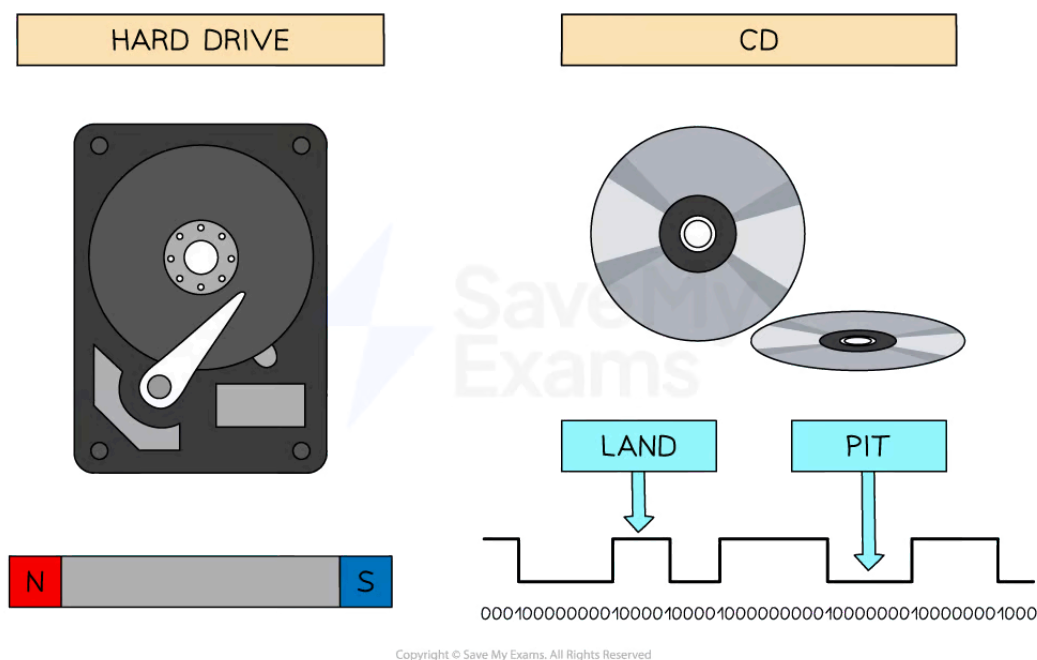


Your notes

Why Computers Use Binary

Why does data have to be converted to binary to be processed by a computer?

- Data is processed in a computer using **logic gates** that only have **two states**
- The binary number system only has two digits (**1/0**), which means each digit can **represent a different state** (1 = on, 0 = off)
- All data must be converted to binary **before** a computer can **understand** and **process** it
- Converting data to binary allows computers to process it at an incredible **speed**, perform complex **calculations** and **store** vast amounts of data **efficiently**



- Secondary storage is a great example of the process
 - Magnetic hard drives** use **North** and **South polarity** to represent a 1 or a 0
 - In **Optical disks**, light hitting a flat area (**land**) is interpreted as a 1 and light hitting a bump (**pit**) is interpreted as a 0

- Take an example of driving a car
 - When driving a car the accelerator pedal is used to increase the cars speed
 - If a car was accelerating from 50mph to 100mph the increase would be gradual
 - In a computer system, the car is doing either 50mph (0) or 100mph (1), there is no in-between
 - Trying to change the computer system so that it has more options would be less efficient and require more complex parts for the computer to understand



Worked Example

Explain why computers process data in binary format [2]

Answer

- Computers process data using logic gates... [1]
- ... that can only have two states (1/0) [1]



Your notes



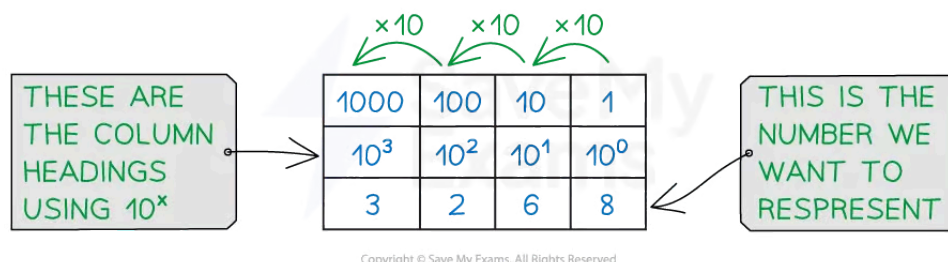
Your notes

Number Systems

The Denary, Binary & Hexadecimal Number Systems

What is denary?

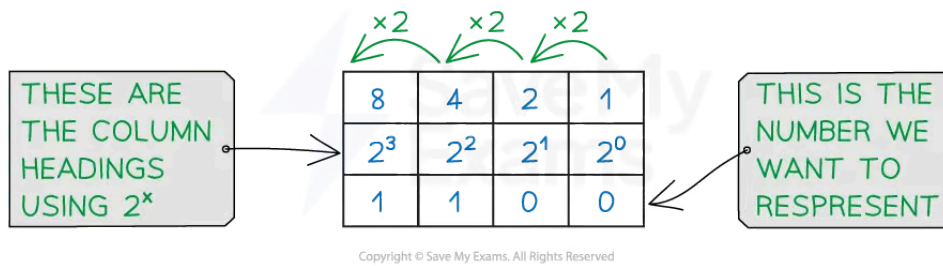
- Denary is a number system that is made up of **10 digits (0–9)**
- Denary is referred to as a **base-10** number system
- Each digit has a weight factor of **10 raised to a power**, the rightmost digit is 1s (10^0), the next digit to the left 10s (10^1) and so on
- Humans use the denary system for **counting, measuring** and **performing maths calculations**
- Using combinations of the 10 digits we can represent any number



- In this example, $(3 \times 1000) + (2 \times 100) + (6 \times 10) + (8 \times 1) = 3268$
- To represent a bigger number we **add more digits**

What is binary?

- Binary is a number system that is made up of **two digits (1 and 0)**
- Binary is referred to as a **base-2** number system
- Each digit has a weight factor of **2 raised to a power**, the rightmost digit is 1s (2^0), the next digit to the left 2s (2^1) and so on
- Each time a new digit is added, the **column value is multiplied by 2**
- Using **combinations of the 2 digits** we can represent any number



- In this example, **Binary 1100** = $(1 \times 8) + (1 \times 4) = 12$
- To represent bigger numbers we add more **binary digits (bits)**

32,768	16,384	8,192	4,096	2,048	1,024	512	256	128	64	32	16	8	4	2	1
2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0



Examiner Tips and Tricks

In the exam you are expected to be able to work with up to and including **16 bits**

The **largest denary number** that can be represented using **16 bits** is:

- **65,535** (Binary 1111111111111111)

What is hexadecimal?

- Hexadecimal is a number system that is made up of **16 digits**, 10 numbers (**0–9**) and 6 letters (**A–F**)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

- Hexadecimal is referred to as a **base-16** number system
- Each digit has a weight factor of **16 raised to a power**, the rightmost digit is 1s (16^0), the next digit to the left 16s (16^1)
- In GCSE you are required to work with up to and including **2 digit hexadecimal** values



Your notes

16s	1s	
1	3	
1x16	3x1	= 19

- A quick comparison table demonstrates a **relationship** between **hexadecimal** and a **binary nibble**
- One hexadecimal digit can represent **four bits** of binary data

Denary	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C

13	1101	D
14	1110	E
15	1111	F



Your notes



Examiner Tips and Tricks

A common exam mistake is mixing up which letter matches with what number, write out the 16 hexadecimal digits at the start of the exam!



Your notes

Converting Between Binary & Denary

Denary to Binary Conversion

How do you convert denary to binary?

- It is important to **know the process** of converting from denary to binary to understand how computers interpret and process data
- To convert from denary to binary you must start by **writing out a binary number line**
- Find the first column heading with a **value larger than the denary value** you are converting
- Write down **each column heading to the right (not including the largest heading)** until you reach 1

32,768	16,384	8,192	4,096	2,048	1,024	512	256	128	64	32	16	8	4	2	1
--------	--------	-------	-------	-------	-------	-----	-----	-----	----	----	----	---	---	---	---

Example 1

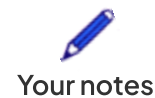
- To convert the denary number **45** to binary, start by **writing out the binary number line**
- The **first column heading larger than 45** is **64**, so the number line would be:

32	16	8	4	2	1

- Start at the **leftmost empty column heading** (32)
- Divide **column heading into denary number** (how many times does 32 fit into 45?)
 - 1 time with 13 remaining

13					
32	16	8	4	2	1
1					

- Repeat with next column heading** (how many times does 16 fit into 13?)
 - 0 times with 13 remaining



13	13				
32	16	8	4	2	1
1	0				

- **Repeat** until all columns have a binary value

13	13	5	1	1	0
32	16	8	4	2	1
1	0	1	1	0	1

- Denary **45** is 101101 in binary (6 bits)



Examiner Tips and Tricks

Always read the question carefully to check for **how many bits are expected** in your answer

- In the example above, a 6 bit answer has been given
- If the question requires an 8 bit answer, add 2 extra 0s to the start of the answer
- Denary 45 is 101101 in **6 bit** binary and 00101101 in **8 bit** binary

Example 2

- To convert the denary number **3059** to binary, start by **writing out the binary number line**
- The **first column heading larger than 3059** is **4096**, so the number line would be:

2,048	1,024	512	256	128	64	32	16	8	4	2	1

- Start at the **leftmost empty column heading** (2048)
- Divide **column heading into denary number** (how many times does 2048 fit into 3059?)
 - 1 time with 1011 remaining



Your notes

1011											
2,048	1,024	512	256	128	64	32	16	8	4	2	1
1											

- Repeat with next column heading (how many times does 1024 fit into 1011?)

- 0 times with 1011 remaining

1011	1011										
2,048	1,024	512	256	128	64	32	16	8	4	2	1
1	0										

- Repeat until all columns have a binary value

1011	1011	499	243	115	51	19	3	3	3	1	0
2,048	1,024	512	256	128	64	32	16	8	4	2	1
1	0	1	1	1	1	1	1	0	0	1	1

- Denary **3059** is 101111110011 in binary (12 bits)

Binary to Denary Conversion

- To convert from binary to denary, count how many bits make up the value
- Write out the column headings for the number of bits given from right to left
- Add together any column heading with a value of 1 in the column

Example 1 (4 bits)

- To convert the binary number 1011 to denary, start by **writing out the binary headings** from right to left

8	4	2	1
---	---	---	---



Your notes

--	--	--	--

- Write in the binary digits under the headings from left to right

8	4	2	1
1	0	1	1

- Add together **any column heading with a 1 under it**
- $(1 \times 8) + (1 \times 2) + (1 \times 1) = 11$
- Binary 1011 is **11** in denary



Examiner Tips and Tricks

If you are converting from binary to denary and the binary number ends in 1, the denary answer must be an odd number!

Example 2 (8 bits)

- To convert the binary number 01100011 to denary, start by **writing out the binary headings** from right to left

128	64	32	16	8	4	2	1

- Write in the binary digits under the headings from left to right

128	64	32	16	8	4	2	1
0	1	1	0	0	0	1	1

- Add together **any column heading with a 1 under it**
- $(1 \times 64) + (1 \times 32) + (1 \times 2) + (1 \times 1) = 99$
- Binary 01100011 is **99** in denary

Example 3 (14 bits)

- To convert the binary number 01110001110100 to denary, start by **writing out the binary headings** from right to left

8,192	4,096	2,048	1,024	512	256	128	64	32	16	8	4	2	1
0	1	1	1	0	0	0	1	1	1	0	1	0	0

- Add together **any column heading with a 1 under it**
- $(1 \times 4096) + (1 \times 2048) + (1 \times 1024) + (1 \times 64) + (1 \times 32) + (1 \times 16) + (1 \times 4) = 7284$
- Binary 01110001110100 is **7284** in denary



Your notes



Your notes

Converting Between Hexadecimal & Denary

Denary to Hexadecimal Conversion

How do you convert denary to hexadecimal?

Method 1 (denary to binary to hexadecimal)

- To convert the denary number **28** to hexadecimal, start by converting the [denary number to binary](#)

128	64	32	16	8	4	2	1
0	0	0	1	1	1	0	0

- Split the 8 bit binary number into **two nibbles** as shown below

8	4	2	1		8	4	2	1
0	0	0	1		1	1	0	0

- Convert each nibble to its denary value
- 0001 = 1** and **1100 = 12**
- Using the [comparison table](#), the denary value 1 is also 1 in hexadecimal whereas denary value 12 is represented in hexadecimal as C
- Denary **28** is **1C** in hexadecimal

Method 2 (divide by 16)

- To convert the denary number 163 to hexadecimal, start by **dividing the denary** value by 16 and recording the **whole times** the number goes in and the **remainder**
- $163 \div 16 = 10$ remainder 3
- In hexadecimal the **whole number = digit 1** and the **remainder = digit 2**
- Digit 1 = 10 (A)
- Digit 2 = 3
- Denary **163** is **A3** in hexadecimal



Your notes

Hexadecimal to Denary Conversion

How do you convert hexadecimal to denary?

Method 1 (hexadecimal to binary to denary)

- To convert the hexadecimal number **B9** to denary, take each hexadecimal digit and convert it from its **denary value** to **4 bit binary** (nibble)

B (11)					9			
8	4	2	1		8	4	2	1
1	0	1	1		1	0	0	1

- Join** the two nibbles to make an **8 bit number** (byte)
- Convert from [binary to denary](#)

128	64	32	16	8	4	2	1
1	0	1	1	1	0	0	1

- $(1 \times 128) + (1 \times 32) + (1 \times 16) + (1 \times 8) + (1 \times 1) = 185$
- Hexadecimal **B9** is **185** in denary

Method 2 (multiply by 16)

- To convert the hexadecimal number **79** to denary, start by **multiplying the first hexadecimal digit by 16**
- $7 \times 16 = 112$
- Add digit 2** to the result
- $112 + 9 = 121$
- Hexadecimal **79** is **121** in denary



Examiner Tips and Tricks

Remember that the exam is non-calculator, if you are not confident multiplying and dividing by 16 then use method 1 on both conversions



Your notes



Your notes

Converting Between Hexadecimal & Binary

Binary to Hexadecimal Conversion

How do you convert from binary to hexadecimal?

- It is important before revising how to convert from binary to hexadecimal and vice versa that you fully understand the [binary](#) and [hexadecimal](#) number systems.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Example 1

- To convert the binary number 10110111 to **hexadecimal**, first split the 8 bit number into 2 binary **nibbles**

8	4	2	1		8	4	2	1
1	0	1	1		0	1	1	1

- For each nibble, convert the binary to it's denary value
- $(1 \times 8) + (1 \times 2) + (1 \times 1) = 11$ (B)
- $(1 \times 4) + (1 \times 2) + (1 \times 1) = 7$
- Join them together to make a 2 digit hexadecimal number
- Binary 10110111 is **B7** in hexadecimal

Example 2

- To convert the binary number 00111001 to **hexadecimal**, first split the 8 bit number into 2 binary nibbles

8	4	2	1		8	4	2	1
0	0	1	1		1	0	0	1

- For each nibble, convert the binary to it's denary value
- $(1 \times 2) + (1 \times 1) = 3$



Your notes

- $(1 \times 8) + (1 \times 1) = 9$
- Join them together to make a 2 digit hexadecimal number
- Binary 00111001 is **39** in hexadecimal

Hexadecimal to Binary Conversion

How do you convert from hexadecimal to binary?

Example 1

- To convert the hexadecimal number **5F** to binary, first **split the digits apart** and convert each to a binary **nibble**

8	4	2	1	
0	1	0	1	= 5

8	4	2	1	
1	1	1	1	= 15 (F)

- Join the 2 binary nibbles together to create an 8 bit binary number

128	64	32	16	8	4	2	1
0	1	0	1	1	1	1	1

- Hexadecimal **5F** is **01011111** in binary

Example 2

- To convert the hexadecimal number **26** to binary, first **split the digits apart** and convert each to a binary **nibble**

8	4	2	1	
0	0	1	0	= 2



Your notes

8	4	2	1	
0	1	1	0	= 6

- Join the 2 binary nibbles together to create an 8 bit binary number

128	64	32	16	8	4	2	1
0	0	1	0	0	1	1	0

- Hexadecimal **26** is **00100110** in binary



Your notes

Uses of Hexadecimal

Uses of Hexadecimal

Why is hexadecimal used?

- In Computer Science hexadecimal is often preferred when working with **large values**
- It takes **fewer digits to represent a given value** in hexadecimal than in binary
 - 1 hexadecimal digit corresponds 4 bits (one nibble) and can represent 16 unique values (0–F)
- It is **beneficial** to use hexadecimal over binary because:
 - The more bits there are in a binary number, the **harder it makes for a human to read**
 - Numbers with more bits are more **prone to errors** when being copied
- Examples of where hexadecimal can be seen:
 - **MAC addresses**
 - **Colour codes**
 - **URLs**

MAC addresses

- A typical MAC address consists of **12 hexadecimal digits**, equivalent to **48 digits** in binary
 - **AA:BB:CC:DD:EE:FF**
 - **10101010:10111011:11001100:11011101:11101110:11111111**
- **Writing down or performing calculations** with 48 binary digits makes it **very easy to make a mistake**

```
Command Prompt  x + | v
Ethernet adapter Ethernet 4:

    Connection-specific DNS Suffix.: cable.virginm.net
    Description.: Realtek PCIe 2.5GbE Family Controller
    Physical Address.: 2C-F0-7C-A7-98
    DHCP Enabled.: Yes
    Autoconfiguration Enabled.: Yes
```

Copyright © Save My Exams. All Rights Reserved

Colour codes

- A typical hexadecimal colour code consists of **6 hexadecimal digits**, equivalent to **24 digits** in binary
 - #66FF33** (green)
 - 01000010:11111111:00110011**



51.102.255
#3366FF



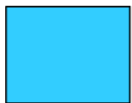
102.51.255
#6633FF



204.51.255
#CC33FF



255.51.204
#FF33CC



51.204.255
#33CCFF



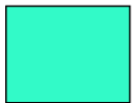
0.61.245
#003DF5



0.46.184
#002EB8



255.51.102
#FF3366



51.255.204
#33FFCC



184.138.0
#B88A00



245.184.0
#F5B800



255.102.51
#FF6633



51.255.102
#33FF66



102.255.51
#66FF33



204.255.51
#CCFF33



255.204.51
#FFCC33



Your notes

URL's

- A URL can only contain **standard characters** (a-z and A-Z), **numbers** (0-9) and some **special symbols** which is enough for basic web browsing

- If a URL needs to include a character **outside of this set**, they are converted into a **hexadecimal code**
- Hexadecimal codes included in a URL are prefixed with a % sign



Your notes



Your notes

Binary Addition

Adding Positive 8-bit Binary Integers

What is binary addition?

- Binary addition is the process of **adding together two binary integers** (up to and including 8 bits)
- To be successful there are **5 golden rules** to apply:

Binary Addition	Binary Answer	Working		
0 + 0 =	0	1s		
		0	= 0	
0 + 1 =	1	1s		
		1	= 1	
1 + 0 =	1	1s		
		1	= 1	
1 + 1 =	10	2s	1s	
		1	0	= 2
1 + 1 + 1 =	11	2s	1s	
		1	1	= 3

- Like denary addition, start from the rightmost digit and move left
- Carrying over occurs when the **sum of a column is greater than 1**, passing the excess to the next left column



Your notes

Example 1

- Add together the binary values **1001** and **0100**

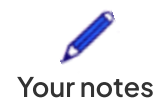
8	4	2	1	+
1	0	0	1	
0	1	0	0	
				C

- Starting from right to left, add the two binary values together **applying the 5 golden rules**
- If your answer has **2 digits**, place the rightmost digit in the column and **carry the remaining digit** to the next column on the left
- In this example, start with $1+0$, $1+0 = 1$, so place a 1 in the column

8	4	2	1	+
1	0	0	1	
0	1	0	0	
				C
			1	

- Repeat until **all columns have a value**

8	4	2	1	+



1	0	0	1	
0	1	0	0	
				C
1	1	0	1	

- The sum of adding together binary **1001** (9) and **0100** (4) is **1101** (13)



Examiner Tips and Tricks

Make sure any carried digits are clearly visible in your answer, there are marks available for working.
Carries can be put above or below in the addition

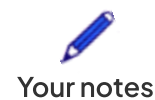
Example 2

- Add together the binary values **00011001** and **10001001**

128	64	32	16	8	4	2	1	+
0	0	0	1	1	0	0	1	
1	0	0	0	1	0	0	1	
								C

- Starting from right to left, add the two binary values together **applying the 5 golden rules**
- If your answer has **2 digits**, place the rightmost digit in the column and **carry the remaining digit** to the next column on the left
- In this example, start with $1+1$, $1+1=10$, so place a 0 in the column and carry the 1 to the next column

128	64	32	16	8	4	2	1	+
-----	----	----	----	---	---	---	---	---



0	0	0	1	1	0	0	1	
1	0	0	0	1	0	0	1	
						1		C
							0	

- Repeat until **all columns have a value**

128	64	32	16	8	4	2	1	+
0	0	0	1	1	0	0	1	
1	0	0	0	1	0	0	1	
		1	1			1		C
1	0	1	0	0	0	1	0	

- The sum of adding together binary **00011001** (25) and **10001001** (137) is **10100010** (162)

Overflow & Binary Addition

What is an overflow error?

- An overflow error occurs when the result of a binary addition **exceeds the available bits**
- For example, if you took binary **11111111** (255) and tried to add **00000001** (1) this would cause an overflow error as the result would need a **9th** bit to represent the answer (256)

256	128	64	32	16	8	4	2	1	+
	1	1	1	1	1	1	1	1	
	0	0	0	0	0	0	0	1	
1	1	1	1	1	1	1	1		C

	0	0	0	0	0	0	0	0	
--	---	---	---	---	---	---	---	---	--



Your notes



Examiner Tips and Tricks

When starting a binary addition question, always look at the question that comes after. If it asks you to name what problem has been caused, you know the binary addition question must cause an overflow error and therefore mean a carried bit that does not fit in to the answer



Your notes

Binary Shifts

Binary Shifts

What is a logical binary shift?

- A logical binary shift is how a computer system performs **basic multiplication and division** on non-negative values (0 and positive numbers)
- Binary digits are **moved left** or **right** a set number of times
- A left shift **multiplies** a binary number by 2 (x2)
- A right shift **divides** a binary number by 2 (/2)
- A shift can move more than one place at a time, the principle remains the same
- A left shift of 2 places would multiply the original binary number by 4 (x4)

How do you perform a logical left shift of 1?

- Here is the binary representation of the denary number 40

128	64	32	16	8	4	2	1
0	0	1	0	1	0	0	0

- To perform a left logical binary shift of 1, we move each bit 1 place to the left
- The digit in the 128 column (**MSB**) will move left causing an **overflow error**
- The 1 column becomes empty so is filled with a 0

128	64	32	16	8	4	2	1	
	0	1	0	1	0	0	0	= 40
0	1	0	1	0	0	0	0	= 80

- The original binary representation of denary **40** (32+8) was **multiplied by 2** and became **80** (64+16)

How do you perform a logical left shift of 2?

- Here is the binary representation of the denary number 28

128	64	32	16	8	4	2	1
0	0	0	1	1	1	0	0

- To perform a left binary shift of 2, we move each bit 2 places to the left
- The digit in the 128 (**MSB**) and 64 column will move left causing an **overflow error**
- The 1 and 2 column become empty so are filled with a 0

128	64	32	16	8	4	2	1	
		0	1	1	1	0	0	= 28
0	1	1	1	0	0	0	0	= 112

- The original binary representation of denary **28** ($16+8+4$) was **multiplied by 4** and became **112** ($64+32+16$)

How do you perform a logical right shift of 1?

- Here is the binary representation of the denary number 40

128	64	32	16	8	4	2	1
0	0	1	0	1	0	0	0

- To perform a right binary shift of 1, we move each bit 1 place to the right
- The digit in the 1 column (**LSB**) will move right causing an **underflow error**
- The 128 column becomes empty so is filled with a 0

128	64	32	16	8	4	2	1	
0	0	1	0	1	0	0		= 40
0	0	0	1	0	1	0	0	= 20



Your notes

- The original binary representation of denary **40** ($32+8$) was **divided by 2** and became **20** ($16+4$)

How do you perform a logical right shift of 2?

- Here is the binary representation of the denary number 200

128	64	32	16	8	4	2	1
1	1	0	0	1	0	0	0

- To perform a right binary shift of 2, we move each bit 2 places to the right
- The digits in the 1 (**LSB**) and 2 columns will move right causing an underflow error
- The 128 and 64 columns become empty so are filled with a 0

128	64	32	16	8	4	2	1	
1	1	0	0	1	0			= 200
0	0	1	1	0	0	1	0	= 50

- The original binary representation of denary **200** ($128+64+8$) was **divided by 4** and became **50** ($32+16+2$)



Your notes



Your notes

Two's Complement

Two's Complement

What is two's complement?

- In IGCSE Computer Science, two's complement is a method of using **signed binary values** to **represent negative numbers**
- Using two's complement the **left most bit** is designated the most significant bit (**MSB**)
- To represent negative numbers this bit **must equal 1**, turning the column value in to a **negative**
- Working with **8 bits**, the **128** column becomes **-128**

-128	64	32	16	8	4	2	1	
1	1	1	1	1	1	1	1	= -1

- In the example above to represent -1, **add column values** with a **1** to the **MSB**
 - MSB** (-128)
 - Add 64** ($-128 + 64 = -64$)
 - Add 32** ($-64 + 32 = -32$)
 - Add 16** ($-32 + 16 = -16$)
 - Add 8** ($-16 + 8 = -8$)
 - Add 4** ($-8 + 4 = -4$)
 - Add 2** ($-4 + 2 = -2$)
 - Add 1** ($-2 + 1 = -1$)
- The two's complement representation of -1 is 11111111

Quick two's complement conversion

- To represent -76
- Write out the **positive version of the number**

128	64	32	16	8	4	2	1	
-----	----	----	----	---	---	---	---	--



Your notes

0	1	0	0	1	1	0	0	= 76
---	---	---	---	---	---	---	---	------

- Starting from the **least significant bit** (right most column), copy out the binary values **up to and including the first 1**

-128	64	32	16	8	4	2	1
					1	0	0

- For the remaining digits, **invert them** (0s to 1s/1s to 0s)

-128	64	32	16	8	4	2	1
1	0	1	1	0	1	0	0

- $-128 + 32 + 16 + 4 = -76$
- The two's complement representation of -76 is 10110100